

# TabUF 数据生成技术文档

从总体到 Episode：一篇会跟着我们一起长的说明书

Heyang Gong      文献调研（协作草稿）

2026 年 8 月 14 日    草稿 v0.1

这不是最终 API 规格。API 会跟着这篇文档一起改。

仓库：<https://github.com/1587causalai/tabuf-episode-api>

## 目录

|     |                                |   |
|-----|--------------------------------|---|
| 1   | 先把话说清楚                         | 2 |
| 2   | 我们到底要喂给网络什么                    | 2 |
| 3   | 为什么必须先抽总体，再填格子                 | 3 |
| 3.1 | 行不是一个新世界                       | 3 |
| 3.2 | 一个小例子，把这句话听进去                  | 3 |
| 4   | 三层流水线：Prior / World / Compiler | 4 |
| 4.1 | Prior：抽总体，不抽考题                 | 4 |
| 4.2 | World：按响应律把格子填满                | 4 |
| 4.3 | Compiler：出题，不改世界               | 5 |
| 5   | 第零版生成律，逐步拆开                    | 5 |
| 6   | 调用方能看见什么，不能看见什么                | 6 |
| 7   | 当前 API 草稿（会变）                  | 6 |
| 8   | 刻意关掉的东西                        | 7 |
| 9   | 一个完整的数值散步                      | 8 |
| 10  | 以后会变复杂的方向（只指路，不开工）             | 8 |
| 11  | 下一轮我们要一起拍的问题                   | 8 |

## 1 先把话说清楚

### 这一节在讲什么

我们现在最缺的不是又一个网络结构，而是**可训练的 episode 从哪来**。这篇文档只做一件事：把「数据怎么生成」讲到你可以抬杠、可以改、可以写成代码的程度。HTTP 路径、JSON 字段名、端口号，都是后话。

你已经有一个能跑的 `POST /v0/episodes`。它远远不够，原因很简单：它把第零版生成律塞进了一个临时接口里，但没有把**生成这件事本身**讲清楚。接口可以换，生成语义不能糊。

所以我们换一个工作方式。以后每一轮讨论，都先改这篇文档，再改代码。文档里用三种颜色盒子：

- **绿：已锁定**。改它等于改项目方向，要明确说「我反悔了」。
- **黄：还没锁**。这一轮可以拍，也可以先挂着。
- **灰：以后再做**。不是忘了，是第零版故意关掉，免得引擎一开始就长歪。

### 已锁定

第零版只生成**观测世界**的  $\text{Unit} \times \text{Feature}$  格子，再切成格子级的上下文  $C$  与查询  $Q$ 。调用方拿到的是 episode，不是 DAG，也不是潜变量  $\mathbf{u}_i$ 。

## 2 我们到底要喂给网络什么

表格基础模型的训练对象，常常被说成「一张表」。这对 TabUF 不够用。一张完整的表，更像一次人口普查的结果；网络真正要学的，是：**在已经看见一部分格子之后，去填另一部分格子**。

所以训练样本不是表，是 **episode**：

$$\text{episode} = (Y^{\text{in}}, M^C, M^Q, y^Q). \quad (1)$$

记号先定下来，后面一直用：

- $n$ ：这一次实现里有多少个 unit（行）。
- $d$ ：有多少个 feature（列）。
- 格子  $(i, j)$ ：第  $i$  个 unit 在第  $j$  个 feature 上的取值  $Y_{ij}$ 。
- $C$ ：上下文格子。网络可以看见这些值。
- $Q$ ：查询格子。网络要预测这些值。第零版只训回归臂  $R$ 。
- $Y^{\text{in}}$ ：把  $Q$  位置挖空之后的输入表。
- $y^Q$ ：按  $M^Q$  拉成向量的查询真值。

$$C \cap Q = \emptyset, \quad C \cup Q = \{1, \dots, n\} \times \{1, \dots, d\}. \quad (2)$$

第零版没有「自然缺失」。格子要么在  $C$ ，要么在  $Q$ 。缺的那一块是我们故意盖住的考题，不是世界自己掉的数据。

为什么是格子级，而不是整行、整列当 query？因为 TabUF 要能回答的，不只是「这个人缺一个标签」，还包括「这个特征在这些人身会上会是什么」。推荐场景里，这几乎是默认形态：人  $\times$  物品的一部分被看见，另一部分要估。

#### 已锁定

训练样本是格子级  $C/Q$  的 episode，不是整张表，也不是一个 DAG。 $C \cap Q = \emptyset$ ，第零版  $C \cup Q$  覆盖全表。

### 3 为什么必须先抽总体，再填格子

这一节是整篇文档的脊柱。如果你只记住一件事，记住它。

#### 3.1 行不是一个新世界

常见的表格合成器（包括不少 SCM / 树 SCM 预训练引擎）骨子里是：**每一行独立地再抽一遍机制**。行与行之间可以相关，但「这一行是谁」往往没有单独的身份。

DiscoSCM 不这么看。它把随机性劈成两截 [1]：

- $U$ ：unit selection。一次实现  $U = u$  就是一个个体。
- $E$ ：这个个体身上的外生噪声。它独立于  $U$ 。

结构方程是 **unit-specific** 的：

$$v_\ell \leftarrow f_\ell(pa_\ell, e_\ell; u). \quad (3)$$

同一个机制  $f_\ell$ ，换一个  $u$ ，响应可以不同。 $u$  钉住的是个体，不是行号，更不是「又一张新表」。

于是生成顺序被钉死了：

1. 先有一个总体  $\Pi$ ，从中实现  $n$  个 unit，得到  $\{\mathbf{u}_i\}_{i=1}^n$ 。
2. 再对这些已经实现的人，按 (3) 填他们在各个 feature 上的取值。

`sample_population` 然后 `fill_grid`。不能倒过来，也不能把「抽一行噪声」误写成「抽一个新总体」。

#### 3.2 一个小例子，把这句话听进去

假设总体里有两种人。第一种人对折扣很敏感，第二种人几乎不买账。如果你每一行都重新抽一套机制，表会看起来很热闹，但网络学到的是「行与行之间的噪声花样」，不是「同一种人在不同列上应当怎么响应」。

TabUF 要的是后者。推荐、Uplift、个性化决策，问的都是：**这个 unit 换一个 feature / 换一个处理，会怎样**。没有钉住的  $u$ ，后面的反事实臂根本没有主语。

**已锁定**

先从总体  $\Pi$  实现  $\{\mathbf{u}_i\}$ ，再用 unit-specific response law 填格子。 $\mathbf{u}_i$  是个体，不是 row-id。不要把每一张表、每一行当成一个新总体。

**以后再做，现在故意不做**

第零版不做干预，不重抽反事实噪声  $\mathbf{E}(\mathbf{x})$ 。那是 DiscoSCM Layer 3 的能力，也是以后 S 臂 / 反事实 episode 的事。现在只抽一次观测噪声  $\mathbf{E}$ ，得到一个观测世界。

## 4 三层流水线：Prior / World / Compiler

把生成想成三条传送带，比想成「一个函数吐 JSON」清楚得多。网络不准上传送带。传送带只负责世界和考题。

### 4.1 Prior：抽总体，不抽考题

Prior 决定「有哪些人、列的方向是什么」。第零版 Prior 非常瘦：

$$\mathbf{u}_i \sim \mathcal{N}(0, I_k), \quad i = 1, \dots, n, \quad (4)$$

$$\mathbf{w}_j \sim \mathcal{N}(0, I_k), \quad j = 1, \dots, d, \quad (5)$$

然后（可选）把  $\mathbf{W} \in \mathbb{R}^{d \times k}$  的每一列做  $\ell_2$  归一，让  $k$  个潜在轴在特征空间里尺度差不多。 $\{\mathbf{u}_i\}$  合起来就是这一次实现的总体切片； $\mathbf{W}$  是这个世界上共享的 feature 方向。

这里的  $k$  叫 `unit_dim`。它不是「网络宽度」，是个体嵌入的生成维数。

### 4.2 World：按响应律把格子填满

观测世界第零版只承认一条线性因子律：

$$Y_{ij} = \langle \mathbf{u}_i, \mathbf{w}_j \rangle + e_{ij}, \quad e_{ij} \sim \mathcal{N}(0, \sigma^2). \quad (6)$$

矩阵写法更干净：

$$\mathbf{Y} = \mathbf{U} \mathbf{W}^\top + \mathbf{E}, \quad \mathbf{U} \in \mathbb{R}^{n \times k}, \mathbf{W} \in \mathbb{R}^{d \times k}, \mathbf{E} \in \mathbb{R}^{n \times d}. \quad (7)$$

这不是 MiniSCM，也不是 TabICL 的 ExtraTrees 生成器，更不是把 TabPFN 未公开先验包一层皮。它是 DiscoSCM 语义在表格格子上的**最瘦实现**：个体  $\mathbf{u}_i$  与列方向  $\mathbf{w}_j$  的内积，加上一次观测噪声。

为什么从这么瘦的东西开始？因为我们要先验证三件事能不能同时成立：

1. 同一个人在不同列上的取值，真的被同一个  $\mathbf{u}_i$  串起来；
2. 同一列在不同人身上的取值，真的被同一个  $\mathbf{w}_j$  串起来；
3. 网络只看见格子，却仍能把查询格填对。

这三件事过不了，后面换更漂亮的  $f_j(\mathbf{u}_i)$  没有意义。

### 4.3 Compiler: 出题, 不改世界

World 给出完整的  $\mathbf{Y}$ 。Compiler 不碰生成律, 只出题:

1. 在  $n \cdot d$  个格子里随机抽约  $q$  比例进  $Q$ , 其余进  $C$ 。
2. 做  $Y^{\text{in}}$ : 把  $Q$  位置写成「看不见」(实现上是 NaN / JSON null)。
3. 把  $Q$  上的真值按行优先拉成  $y^Q$ 。
4. 默认丢掉  $\mathbf{U}$  和  $\mathbf{W}$ 。

Compiler 还可以顺手带一个  $S$  臂诊断位  $\alpha$  (例如「这一格是否该被当成反事实查询」)。第零版可以发  $\alpha$ , 但不准拿它和  $R$  损失平均成一个头。现在只训  $R$ 。

#### 已锁定

流水线固定为 Prior  $\rightarrow$  World  $\rightarrow$  Compiler。网络不进生成器。 $R$  与  $S$  即使将来同 episode, 也不许静默平均成一个头。真目标表上从第一天就冻  $\theta$ , 合成引擎只负责预训练 episode。

#### 还没锁, 下一轮一起拍

下列细节还没锁, 正好是后面几轮要一起拍的:

- 同一个总体能否跨多个 episode 复用 (先抽一个大  $N$  的池子, 再每次取  $n$  行)?
- $\mathbf{W}$  是每个 episode 独立抽, 还是一个「世界」共享、episode 只换 query 掩码?
- $q$  固定 0.15, 还是对  $q$  本身做 curriculum?
- 列归一化是默认开, 还是变成 Prior 的一个开关?

## 5 第零版生成律, 逐步拆开

把 (6) 再讲一遍, 这次按代码会执行的顺序。

**第一步: 实现总体。** 抽  $\mathbf{U} \in \mathbb{R}^{n \times k}$ 。行  $\mathbf{u}_i$  就是第  $i$  个人。不要在注释里写 “sample a table of rows”。写 “sample a population of units”。

**第二步: 实现列方向。** 抽  $\mathbf{W} \in \mathbb{R}^{d \times k}$ 。行  $\mathbf{w}_j$  是第  $j$  列在个体空间里的方向。列归一化 (如果开) 是

$$\mathbf{w}_{:r} \leftarrow \mathbf{w}_{:r} / \|\mathbf{w}_{:r}\|_2, \quad r = 1, \dots, k. \quad (8)$$

**第三步: 一次观测噪声。**  $\mathbf{E} \sim \mathcal{N}(0, \sigma^2 I)$ , 与  $\mathbf{U}$  独立。这就是 DiscoSCM 里的 factual noise。第零版到此为止, 不再抽  $\mathbf{E}(\mathbf{x})$ 。

**第四步: 填满格子。**  $Y_{ij} = \langle \mathbf{u}_i, \mathbf{w}_j \rangle + e_{ij}$ 。此刻世界已经完成。后面任何掩码都不许回头改  $Y_{ij}$ 。

**第五步: 出题。** 令  $N_{\text{cell}} = nd$ ,  $n_Q = \text{clip}(\text{round}(q \cdot N_{\text{cell}}), 1, N_{\text{cell}} - 1)$ , 均匀无放回抽  $Q$ 。 $C$  是补集。于是一定有至少一个上下文格、至少一个查询格。

**第六步：打包给调用方。** 默认 payload 只有世界的可观察部分和考题，没有潜变量。

### 已锁定

第零版响应律就是 (6)。数值、加性、高斯噪声、观测一次。不把 TabICL / TabPFN 先验混进同一套 epoch。

## 6 调用方能看见什么，不能看见什么

这一节是网络合同。API 字段名以后可以改，合同不能偷偷改。

默认返回：

| 字段           | 形状           | 含义                                 |
|--------------|--------------|------------------------------------|
| y_input      | $n \times d$ | 上下文格是 $Y_{ij}$ ，查询格是空              |
| context_mask | $n \times d$ | $M_{ij}^C = 1$ 当且仅当 $(i, j) \in C$ |
| query_mask   | $n \times d$ | $M_{ij}^Q = 1$ 当且仅当 $(i, j) \in Q$ |
| y_query      | $n_Q$        | $Y$ 在 $Q$ 上按行优先拉直                  |
| shapes       | —            | 上面这些张量的形状，避免客户端猜                   |
| seed         | 标量           | 这一集的名字，可复现                         |

默认不返回  $U$ 、 $W$ 、完整  $Y$ 。debug=true 才附带，而且 debug 输出不准进训练 DataLoader。

为什么这么凶？因为  $u_i$  一旦进网络，模型会走捷径：直接读个体真名，不再从格子里推断「这个人是谁」。预训练就会变成漏题。真表上你也没有  $u_i$  可喂。

### 已锁定

训练路径禁止  $u_i$  进网。Compiler 的张量合同先于 Prior 插件化。小格子 JSON，大格子以后再换 npz，但语义不变。

### 还没锁，下一轮一起拍

张量合同里还没锁的：

- 要不要带列类型 column\_types（第零版全是实数，字段可以先占位）？
- $y^Q$  的拉直顺序：行优先已经写进代码，是否正式写进合同？
- 多 episode 一次返回时，是列表还是 padded batch？
- S 臂诊断  $\alpha$  的形状：格子级，还是 episode 级？

## 7 当前 API 草稿（会变）

把现在仓库里的接口抄在这里，只作为**实现快照**，不是承诺。

GET /health 返回版本号。

POST /v0/episodes 现在接受：

| 参数                      | 第零版默认 | 备注                         |
|-------------------------|-------|----------------------------|
| <code>n_units</code>    | 64    | 这一集实现多少 unit               |
| <code>n_features</code> | 8     | 多少列                        |
| <code>unit_dim</code>   | 4     | $k$ , 个体潜空间维               |
| <code>query_frac</code> | 0.15  | $Q$ 占总格子的比例                |
| <code>sigma</code>      | 0.3   | 观测噪声标准差                    |
| <code>seed</code>       | 0     | 第 $e$ 集用 $\text{seed} + e$ |
| <code>n_episodes</code> | 1     | 单次最多 32, 纯属保护网关            |
| <code>debug</code>      | false | 训练必须 false                 |

这些数字没有神圣性。 $64 \times 8$  只是为了能用手印 JSON。真正训练时会走  $n, d$  curriculum: 先小后大, 再 pad 到本 epoch 的  $\max n \times \max d$ 。

### 还没锁, 下一轮一起拍

API 形态本身就是开放问题, 也是这份文档存在的理由。例如:

- 继续 REST JSON, 还是一开始就按 DataLoader 协议走共享内存 / npz?
- 要不要 `population_id`, 让多次请求钉在同一次总体实现上?
- 版本号是 URL 里的 /v0, 还是 payload 里的 `schema` 字段?
- 失败时如何区分「参数不合法」和「生成器内部崩了」?

我们不假装现在就能选对。先把生成语义聊稳, 接口跟着语义长。

## 8 刻意关掉的东西

写下来, 免得下一周手痒又加回来。

### 以后再做, 现在故意不做

- 干预  $do(\mathbf{x})$ , 以及 counterfactual noise 重抽。
- 自然缺失 (World 级 missingness)。现在的空格只来自 Compiler 出题。
- 类别列、混合类型、文本单元格。
- S 臂训练目标。诊断  $\alpha$  可以留位, 损失不能混。
- 把 TabICL ExtraTrees / 未公开 TabPFN 先验接到同一套 epoch。
- 可插拔节点函数  $f_j$  (MLP、树、周期、跳跃)。Prior 插件化要等张量合同先冻住。
- 把网络、tokenizer、损失写进这个仓库。这是数据引擎, 不是训练引擎。

关掉不是因为它们不重要。恰恰相反: 它们每一个都会逼我们改 World 的语义。语义没稳之前加功能, 等于在流沙上盖楼。

## 9 一个完整的数值散步

假设  $n = 4$ ,  $d = 3$ ,  $k = 2$ ,  $q = 0.25$ ,  $\sigma = 0.3$ ,  $\text{seed}=0$ 。(数字是示意, 不以某次运行的浮点为准。)

1. Prior 抽出 4 个 unit, 每个是  $\mathbb{R}^2$  里的一个点。这 4 个人组成这一集的总体切片。
2. 再抽出 3 个列方向, 同样在  $\mathbb{R}^2$  里。列 1 也许更对齐第一个潜在轴, 列 2 更对齐第二个。
3. 对每个人、每一列做内积, 再加一点点高斯噪声, 得到  $4 \times 3$  的满表。
4. 12 个格子里大约 3 个进  $Q$ 。网络看见另外 9 个, 去猜这 3 个。
5. 调用方拿到挖空的表和三个真值。它不知道这 4 个人的  $\mathbf{u}_i$ , 也不知道三列的  $\mathbf{w}_j$ 。

如果网络真的在学 TabUF 该学的东西, 它必须从那 9 个可见格子里, 同时推断「人」和「列」, 再在交点上给出预测。这就是因子律作为第零版考题的原因: 答案结构清楚, 作弊路径也清楚 (把  $\mathbf{U}$  漏出去立刻满分)。

## 10 以后会变复杂的方向 (只指路, 不开工)

等第零版在合成格子上把  $R$  臂打穿, 再按这条路长, 而不是按心情长:

1. **World 级缺失**。先有世界掉格子, 再在剩下的格子上出题。 $C \cup Q$  不再等于全表。
2. **非线性 unit-specific 律**。 $Y_{ij} = f_j(\mathbf{u}_i) + e_{ij}$ ,  $f_j$  从线性换成 MLP / 周期 / 分段, 但仍是同一个  $u$  贯穿一行。
3. **干预 episode**。对某一列做 *do*, 重抽  $\mathbf{E}(\mathbf{x})$ , Compiler 问反事实格。那时  $S$  臂才有真实的监督。
4. **类别与混合类型**。先锁实数合同, 再扩。
5.  **$n, d$  curriculum 与无限 DataLoader**。生成器永远在线, pad 到本 batch 的最大格子。

每加一层, 先改这篇文档的对应小节, 再改代码。代码不许跑到文档前面去。

## 11 下一轮我们要一起拍的问题

把开放问题收成一张清单。你可以从里面点名先聊哪一条。

1. 一个 episode 是否必须对应一次全新的总体实现? 还是允许 `population_id` 钉住同一批  $\mathbf{u}_i$ , 只换掩码?
2.  $\mathbf{W}$  的生命周期: 每集一抽, 还是一个世界多次出题?
3.  $q = 0.15$  是暂时的手感, 还是写成合同?
4. 列类型字段现在占位, 还是干脆第零版不出现?
5. 无限 DataLoader 是这个仓库的事, 还是训练仓库来拉 API?
6. debug 通道要不要用单独的端口 / 路径, 避免误进训练?



7. 文档本身继续中文活文档，还是同时养一份英文（给以后写论文用）？

我的建议（可以否决）：第 1、2 题下一轮先拍，因为它们决定 Prior 要不要有状态；第 5 题可以晚一点，等生成语义稳了再把运输层做厚。

## 这份文档怎么用

- 同意某一段：说「锁第 X 节」。我会把黄盒子改成绿盒子，再改代码。
- 反对某一段：直接改句子。不要只说「API 再想想」——指出是 Prior、World 还是 Compiler 错了。
- 想加功能：先在「以后再做」里找，确认它不是第零版故意关掉的。如果要提前做，写明为什么现在就必须做。

第零版代码只是这篇文档第 4-6 节的一个临时投影。投影可以砸。语义要留下。

## 参考文献

- [1] Heyang Gong, Chaochao Lu, Yu Zhang. Distribution-consistency Structural Causal Models. 2024. 本地指南：`discoscm/latex/main-guide.tex`。
- [2] Judea Pearl. *Causality*. Cambridge University Press, 2nd edition, 2009.
- [3] Paul W. Holland. Statistics and causal inference. *JASA*, 1986.